大数据实训项目汇报　2026 年 11 月

目标适应症及用药数据分析系统

把六张业务表做成 13 个查询接口与三套分析页面

100 万行
处方与用药明细
单月 200812 全量数据

13 个
查询接口
覆盖 12 张分析图表

3 套
分析页面
首页、适应症页、药品页

第二小组

占竞　李艺淼　付悌安　杨博文　洪嘉伟

服务器 192.168.207.179　主站 83 端口　大屏 8000 端口　Kylin 7070 端口

目录

汇报内容与页码

01	**项目背景** 为什么做、总体思路、数据基础	P03
02	**项目设计** 三个问题的设计、页面与接口、任务对照	P06
03	**项目结果** 系统输出、三页实证、结构对比、数据校验	P11
04	**实施过程与方法** 六个阶段、部署方式、启动日志	P17
05	**技术原理与架构** 数据流向、 Kylin 与 Cube 、引擎切换	P22
06	**组织与管理** 人员分工与五天进度	P25
07	**总结** 完成情况与后续计划	P26

为什么做这个项目

手上有六张业务表，日常分析要回答四类问题，但现在每次都要重新写一遍 SQL

业务上要回答的四类问题

哪些病用药最多	按疾病看处方量与费用，确定重点关注哪些适应症
哪些药覆盖广	按药品看覆盖的医院与城市，判断市场覆盖范围
谁在用什么药	按科室、年龄看用药分布，发现结构差异
用得是否合理	注射剂、基本药物、抗菌药物三项占比是否偏高

现在的工作方式与三个困难

1. 打开数据库，手工写多表关联 SQL
2. 等十几秒拿到结果，复制到 Excel
3. 换一个维度， SQL 重写一遍

关联多	一条分析要连三到六张表， SQL 长，容易漏条件
字段脏	维表文本带回车符，同一科室被拆成两项
响应慢	百万行关联一次十几秒，页面点不动

项目要做的：把上面这些重复劳动固化成 13 个固定接口和三套页面，口径统一、结果可复核

总体解决思路

三个困难对应三项设计，最终落到三套页面上

遇到的困难	我们的做法	达到的效果
关联多， SQL 难写	按疾病与药品、疾病、药品、处方与药品四种组合分组统计，写成四张汇总表，接口直接查表	接口不再扫明细表，最慢 15.26 秒
字段脏，口径不一	六张维表建视图去掉回车符，占比等于分组处方数除以维度总处方数，公式只写在服务层	三个页面同一指标由同一段代码算出
响应慢，页面点不动	取值几十种的字段提前算好，取值成千上万种的字段建索引，前端超时放宽到 60 秒	首页 0.61 秒， 13 个接口全部 200

数据从表到页面经过四步

明细表
六张业务表， 100 万行

→

汇总表
七张聚合表， 10 万级

→

接口
13 个查询接口

→

页面
三套页面， 12 张图

数据基础与项目目标

先确认手上有什么数据，再把目标写成可校验的数字

六张业务表

表名	内容	规模
prescription_diseases	处方与用药明细	1000000 行
t_view_disease	疾病三级分类	42501 条
t_view_drug	药品属性	205971 种
t_view_hospital	医院与城市	全国范围
t_view_section	科室	20 类

单月数据，时间字段 200812 ，年龄 40 至 89 岁，男女接近 1 比 1

四条目标

分析视角 **5 类**

地区与医院、年龄、适应症、药品、科室

查询接口 **13 个**

覆盖三套页面的全部图表

响应时间 **2 秒与 20 秒**

单表 2 秒内，多表关联 20 秒内

部署方式 **一条命令**

拉起全部服务并逐项自检

验收依据：接口全部返回 200 、页面取真实接口、 11 项基准值可复核

问题一与设计：六表关联改为查汇总表

一条关于某类疾病用了哪些药的分析要连三张表，百万行每次现算

问题表现

疾病与药品分在两张维表里，处方明细只存编码。要得到某类疾病用了哪些药、各多少，必须把明细表、疾病表、药品表关联起来再分组，一条 SQL 十几行，漏一个条件结果就偏。

我们的做法

1. **先定下常用组合**
   统计需求集中在四类：疾病合计、药品合计、疾病与药品组合、处方与药品组合
2. **按组合建汇总表**
   每张表只存编码、名称、数量三列，主键唯一，从建表上避免重复计数
3. **接口改成查汇总表**
   先用汇总表取出编码与数量，再与维表关联补名称，不再扫明细表
4. **新需求按同样方式扩展**
   科室用药结构对比也是先建 agg_section_drug ，接口只查汇总结果

做出来的效果

4 张汇总表	覆盖 13 个接口中的 8 个
15.26 秒	最慢接口，此前单次关联需数十秒
99 万行	最大汇总表行数，最小 4 万行

问题二与设计：统一维表与统计口径

同一指标两个人算出两个数，根源在维表脏数据和算法不统一

问题表现

维表里的文本字段带回车符，同一科室被拆成两项；药品表有列错位，剂型值出现在厂商列；同一个人在不同页面把分母取错，占比就算错。

我们的做法

1. **六张维表统一建视图**
   每张维表建一个 t_view_ 视图，字段统一去回车、去空格再对外提供

2. **文本字段限长**
   建汇总表时文本列统一截到 255 位，否则主键建不上，导入直接报错

3. **比率算法只写一处**
   占比的分子分母都按处方数加权，写在 Service 层，各页面调用同一个方法

4. **缺失值统一处理**
   名称缺失填未标注或未分类，并且不参与 Top 榜排序

做出来的效果

一处算法	三套页面同一指标由同一段代码算出
9 个索引	建在清洗后的视图字段上
两类脏数据	回字符与列错位已在视图层处理

问题三与设计：预计算加索引

百万行关联一次十几秒，页面等不起，还要避免出现假数据

问题表现

跨表关联一次要十几秒，页面点开是白屏。前端原来设 8 秒超时，把还在计算的请求当成失败，结果页面上一半是真数据一半是假数据。

我们的做法

1. **取值少的字段交给预计算**
   年龄、性别、年月等维度取值在 2 到 431 种之间，预计算成 256 个组合

2. **取值多的字段建索引**
   医院、科室、药品这类有上千上万种取值的字段不预计算，改在 MySQL 建 9 个索引

3. **前端超时与失败处理**
   超时放宽到 60 秒；接口报错时显示错误提示，不再用假数据顶替

4. **一个配置项切换引擎**
   配置 query.engine 在 kylin 与 mysql 之间切换，业务代码不用改

做出来的效果

0.61 秒	首页概览接口实测
15.26 秒	最慢的跨表关联接口
13 个	接口全部返回 200

页面、图表与接口的对应关系

每个页面回答什么问题，背后调哪个接口，一一对应

分析对象	页面	页面上的图表	接口	数据来源
适应症	适应症页	关联药品组合、科室分布、年龄分布、费用负担	/disease/ 开头 5 个	agg_disease_total
药品	药品页	覆盖城市、覆盖医院、适应症分布、科室分布、用药结构对比	/drug/ 开头 5 个	agg_drug_total
机构	首页	省份分布、城市等级、年龄分布、药品前十	/home/ 开头 4 个	明细表与维表
汇总	独立大屏	省份地图、药品前十、病症前十、金额分布、关系图	Django 5 个接口	同一份 MySQL 库

设计上的三点考虑

- 一个页面只回答一组问题，图表按分析对象分组，避免把十几种图堆在一起
- 接口按对象分前缀，前端按页面路径取数，新增分析只需加一个接口和一个组件
- 页面顶部保留关键词搜索框，输入疾病或药品名即可切换分析目标

与课程实训任务的对照

任务资料要求 20 个分析任务，逐条对照我们的实现方式与所用技术

课程任务	要求的做法	我们的实现	状态
任务 12 至 15 城市等级、年龄比例、药品销额、省份格局	后端走 Kylin 查询，前端 Vue 展示	首页四张图，四个接口	已实现
任务 16 至 19 用药组合、联合用药、年龄分布、科室分布	同上	适应症页四张图，四个接口	已实现
任务 20 至 23 覆盖城市、对应适应症、覆盖医院、科室分布	同上	药品页四张图，四个接口	已实现
任务 24 药品关联规则分析	Pandas 与 Apriori 算法	规则文件已接入大屏，药品名编码待统一	部分实现
任务 25 至 31 Django 大屏与五张图	Django 与 ECharts	大屏五个取数接口，地图与关系图取同一份 MySQL	已实现

三处我们自己加的设计

双通道路由

取值少的走 Cube，取值多的走索引

四张预聚合表

把百万行明细压成可复用的中间结果

科室用药结构对比

三套页面里唯一横向比较结构的表

系统输出总览

首页、适应症页、药品页与独立大屏，四块界面全部取真实接口数据

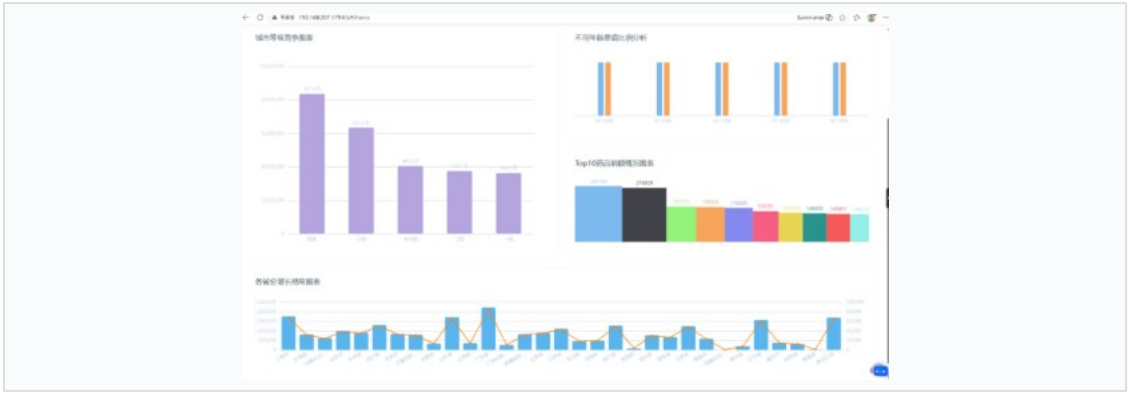


图 1 首页：城市等级、年龄结构、药品销额

采集自 192.168.207.179:83/#/home

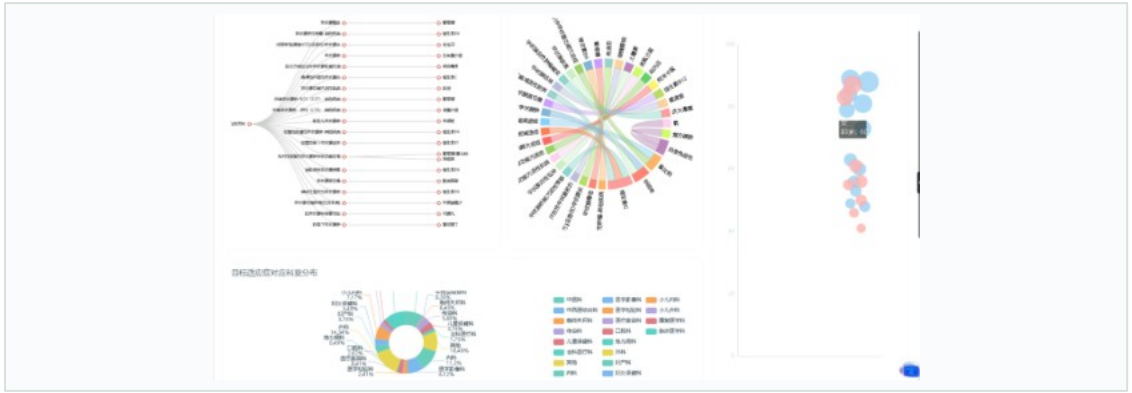


图 2 适应症页：用药组合树与联合用药环

采集自 192.168.207.179:83/#/disease

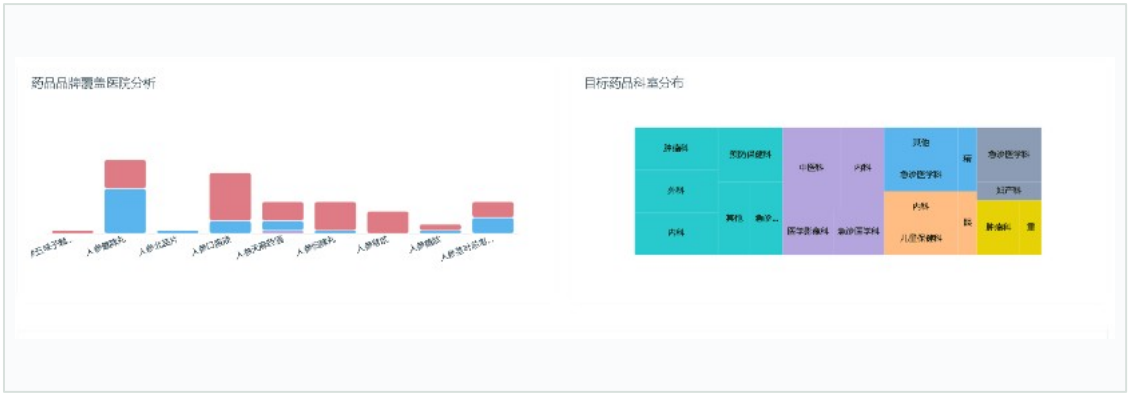


图 3 药品页：覆盖医院与科室分布

采集自 192.168.207.179:83/#/drug

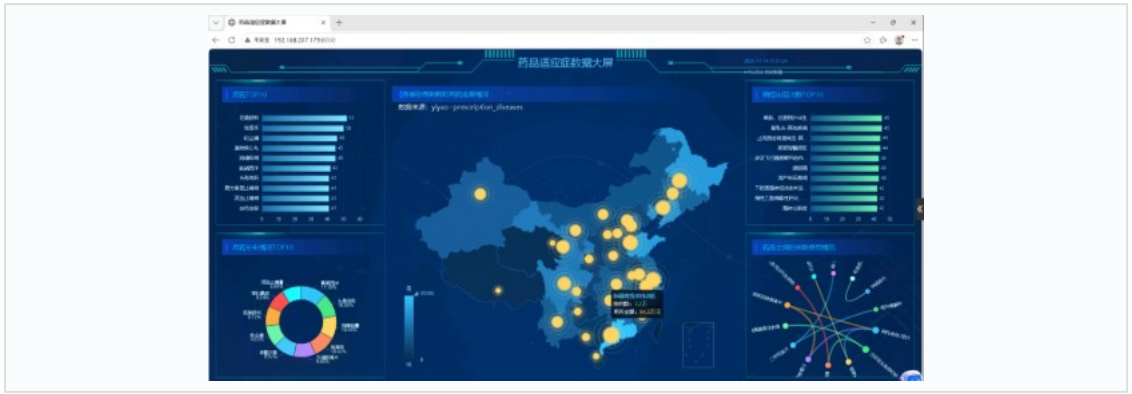


图 4 独立大屏：省份地图与分配指标

采集自 192.168.207.179:8000

首页：四个总量指标加四张分布图

城市等级按销售金额排序，整体呈现四级到一级递减

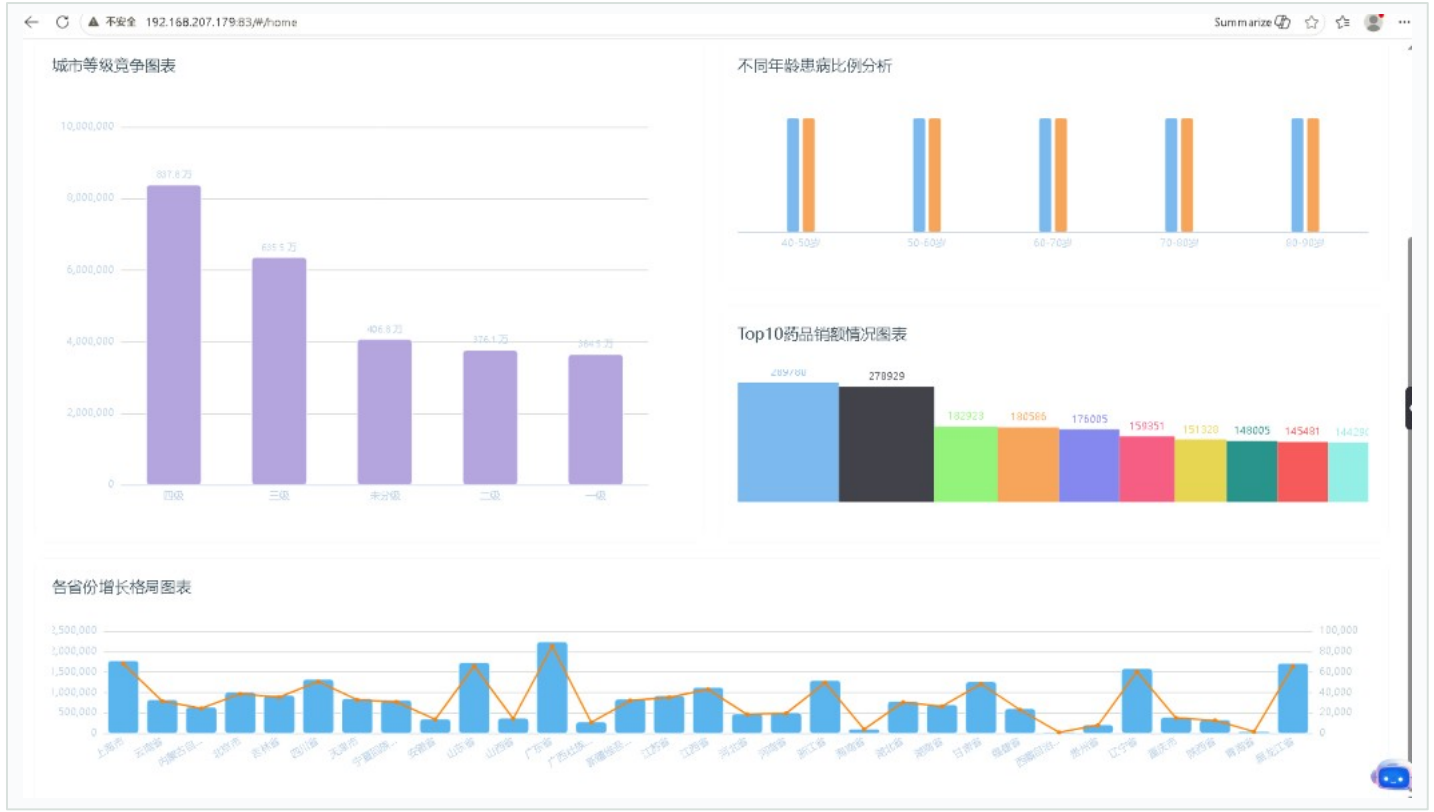


图 5 首页四张分布图，数值取真实接口

采集时间 2026-11-14 17:01， 192.168.207.179:83

100 万行

处方与用药明细

13 个

查询接口，全部 200

0.61 秒

/home/overview 实测

15.26 秒

最慢接口 /disease/union

四级城市 837.8 万元，三级 635.5 万元，一级 364.5 万元，等级越高金额越小

适应症页：某类疾病用了什么药、花多少钱

页面下方是费用负担表，数据来自 /disease/burden

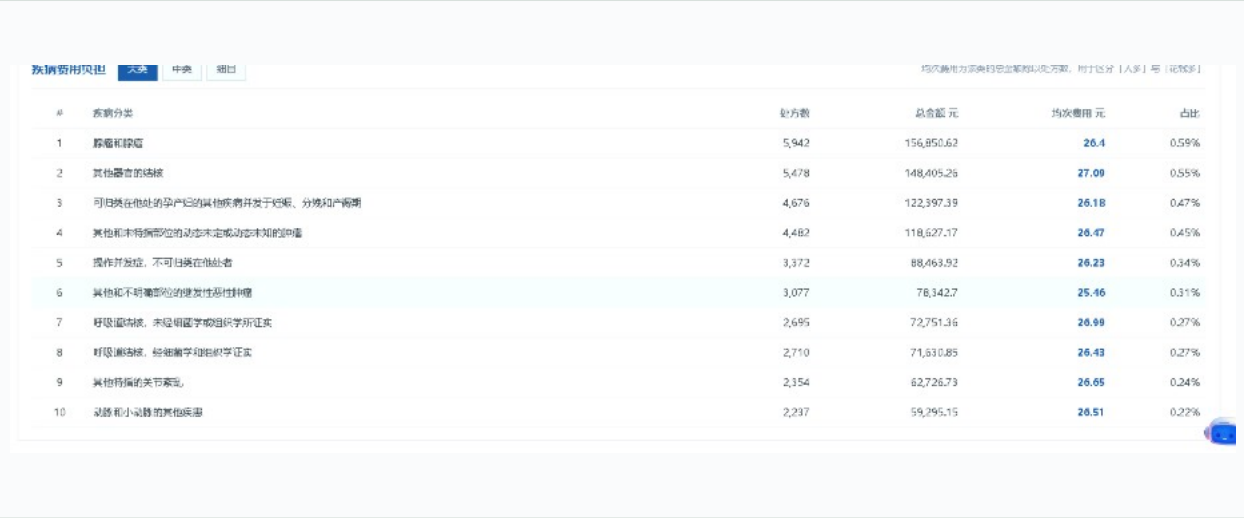


图 6 费用负担表：前十个疾病大类与占比列

采集自 192.168.207.179:83/#/disease

费用负担前五，按总金额排序

疾病大类	处方数	总金额 元	均次 元
腺瘤和腺癌	5,942	156,850.62	26.40
其他器官的结核	5,478	148,405.26	27.09
孕产妇并发其他疾病	4,676	122,397.39	26.18
动态未定或未知的肿瘤	4,482	118,627.17	26.47
操作并发症	3,372	88,463.92	26.23

这三个数怎么算出来的

处方数　该大类下全部处方记录的条数

总金额　该大类各条记录金额求和后保留两位

均次费用　总金额除以处方数

表里能看出的情况

均次费用集中在 26 元附近，排名主要由处方数决定，费用指标区分度低，因此把重点放在三项用药占比上

药品页：新增科室用药结构对比

这一页与另外两页的区别在于，它按科室分组横向比较三项用药占比

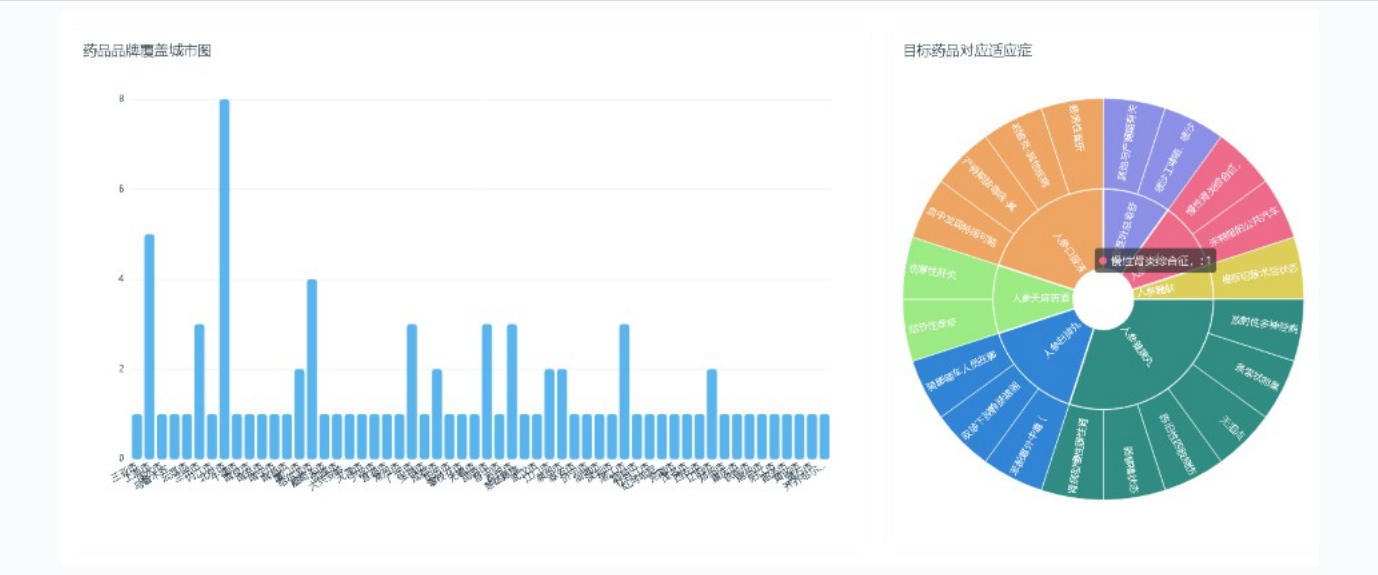


图 7　药品页上半部分：覆盖城市与对应适应症

采集自 192.168.207.179:83/#/drug

下半部分：科室用药结构对比

注射剂占比
剂型含注射的处方数除以该科室处方数

基本药物占比
基本药物标记为是的处方数除以该科室处方数

抗菌药物占比
通用名含抗菌关键词的处方数除以该科室处方数

取处方数前 10 个科室，同表并列

为什么单独留出位置

三套页面里只有这张表把科室当作分组维度横向比较结构，能直接看出哪个科室的注射剂、基本药物与抗菌药物使用比例偏离整体水平。下一张是它的实际运行结果，包括建表耗时、两张表的行数与五个科室的占比。

科室用药结构对比的运行结果

建表用时 1 分 18 秒，两张表共 186.8 万行，三项占比按科室算出

```
[root@master ~]# cd /home/medicine-analysis
[root@master medicine-analysis]# time mysql -uroot -p123456 yiyao < deploy/sql/agg_structure.sql
mysql: [Warning] Using a password on the command line interface can be insecure.

tbl     rows_cnt
section_drug    870702
cat_drug        997681
科室    处方数  注射剂占比      基本药物占比    抗菌药物占比
外科    96988   21.85   20.55   15.02
急诊医学科      86969   21.93   20.61   15.10
其他    82651   21.72   20.27   14.98
内科    77972   21.88   20.43   15.22
肿瘤科  74895   22.09   20.52   15.26

real    1m18.604s
user    0m0.003s
sys     0m0.003s
```

图 8　建表命令与五个科室的三项占比输出

服务器终端实录， 2026-11-14

前五个科室的实测占比

科室	处方数	注射剂	基本药物	抗菌药
外科	96,988	21.85	20.55	15.02
急诊医学科	86,969	21.93	20.61	15.10
其他	82,651	21.72	20.27	14.98
内科	77,972	21.88	20.43	15.22
肿瘤科	74,895	22.09	20.52	15.26

这组数据说明了什么

· 五个科室的注射剂占比在 21.72 到 22.09 之间，跨度 0.37 个百分点；基本药物与抗菌药物同样在 0.4 个百分点以内

· 说明这份数据的科室用药结构高度接近，该表的价值在于监测异常：某个科室一旦偏离这一区间，就能被直接发现

· 原来设想的科室之间差异明显并不成立，因此页面只如实呈现占比，不把它渲染成对比结论

数据一致性校验

总额出现差额时不停留在判断，而是算出差多少、差在哪

事实表直接求和

26,239,620.54

元 一百万行明细

关联维表后求和

26,207,933.86

元 内连接丢失部分记录

差额与原因

31,686.68

元 来自 1,164 行医院编号为空

七张表的行数，取自 06.txt

表名	行数	表名	行数
prescription_diseases	1,000,000	agg_pres_drug	999,999
agg_disease_total	42,501	agg_section_drug	870,702
agg_drug_total	205,971	agg_cat_drug	997,681
agg_disease_drug	999,935	自定义索引	9 个

校验口径

处方表行数、各年龄行数、性别比接近 1 比 1 、索引 9 个、 HDFS 6 个目录，这些指标写成数据基准值文档； dump_baseline.py 可重复导出同一批指标，换人接手时逐项比对即可确认库里的数据没有被改过。

实施流程与阶段产物

六个阶段，每个阶段有明确产物

一	二	三	四	五	六
环境搭建	**数据入仓**	**维度建模**	**接口开发**	**前端联调**	**部署验收**
六个组件端口全通	六张表 100 万行	bi_cube 状态 READY	13 个接口返回 200	12 张图表取真实数	基准值文档与自检

实施中遇到的三个实际情况与处理方式

数据入仓	维度表文本带回车符，分组结果出现重复项	清洗时统一去回车，再重建视图
维度建模	单表 Cube 无法回答跨表关联的 12 个接口	高基数关联改走 MySQL 并建 9 个索引
接口联调	重新编译后新接口仍返回 404	启动脚本比对 jar 与进程时间，自动重启

阶段一二：环境搭建与数据入仓

集群与数据库分别用脚本拉起，端口逐个校验

实际操作命令

```
# 六个组件一次拉起
bash /opt/start-cluster.sh
ss -lnt | grep -E '2181|9000|8088|16010'

# 数据落库与入仓
hdfs dfs -ls /bi_data/mysql
mysql -uroot -p yiyao -e \
  'SELECT COUNT(*) FROM prescription_diseases;'
```

两个数据处理细节

维度表文本字段带回车符，分组时同一名称被拆成两项，清洗时统一去掉

药品表存在后缀列，生成列 drug_code 与业务列 drug 并存，关联时用后者

端口与数据校验结果

2181	ZooKeeper	9000	HDFS	8088	YARN
16010	HBase	9083	Hive 元数据	7070	Kylin
3306	MySQL	83	后端接口	8000	大屏

数据规模

数据项	数量
处方与用药明细	1,000,000 行
疾病维度	42,501 个
药品维度	205,971 种
HDFS 数据目录	6 个

阶段三四：维度建模与接口开发

Cube 负责取值少的字段聚合，接口把结果整理成前端直接可用的结构

bi_cube 实测参数

项目	实测值
Kylin 项目	yiyao
Cube 名称	bi_cube
构建状态	READY
源记录数	1,000,000
Cuboid 数量	256
Cube 体积	2.95 GB
构建耗时	23.13 分钟

维度取年龄、性别、年月、医院、科室、职称、支付方式、数据类型八列

13 个接口实测耗时

接口	秒	接口	秒
/home/overview	0.61	/disease/age	7.20
/home/citylevel	2.12	/disease/section	9.34
/home/age	2.19	/drug/cities	5.20
/home/top10	5.84	/drug/hospital	7.67
/home/province	1.99	/drug/diseases	14.67
/disease/union	15.26	/drug/section	12.98
/disease/drug	15.17		

验收口径

单表不超过 2 秒，多表关联不超过 20 秒， 13 个接口全部满足

部署方式：目录结构与启动自检

脚本放在 /opt 下，可反复执行，不会重复拉起进程

服务器目录结构

/home/medicine-analysis	项目根目录
├ backend/yiyao	服务源码与 jar
├ frontend	Vue 源码与构建产物
├ screen	Django 大屏
├ deploy/sql	建表与聚合脚本
└ deploy/scripts	启停与状态自检

六个组件与两个应用

ZooKeeper 2181 、 HDFS 9000 、 YARN 8088 、 HBase 16010 、 Hive 元数据 9083 、 Kylin 7070

后端 83 与 Django 大屏 8000 ，由同一个脚本检测后拉起

脚本执行的三个动作

步骤一 检测 — 六个集群端口与 MySQL 逐个探测，已在跑的直接跳过

步骤二 拉起 — 后端以 nohup 启动，轮询 /home/overview 直到返回 200

步骤三 自检 — 13 个接口逐个请求，打印状态码与耗时，失败项标红

处理过的一次端口占用问题

重新编译后 83 端口仍在监听，脚本判定无需启动，新接口返回 404 。现在比对 jar 与进程的修改时间， jar 更新就自动重启。

一键启动的实际输出

五阶段自检全部通过，13 个接口逐个打印状态码与耗时

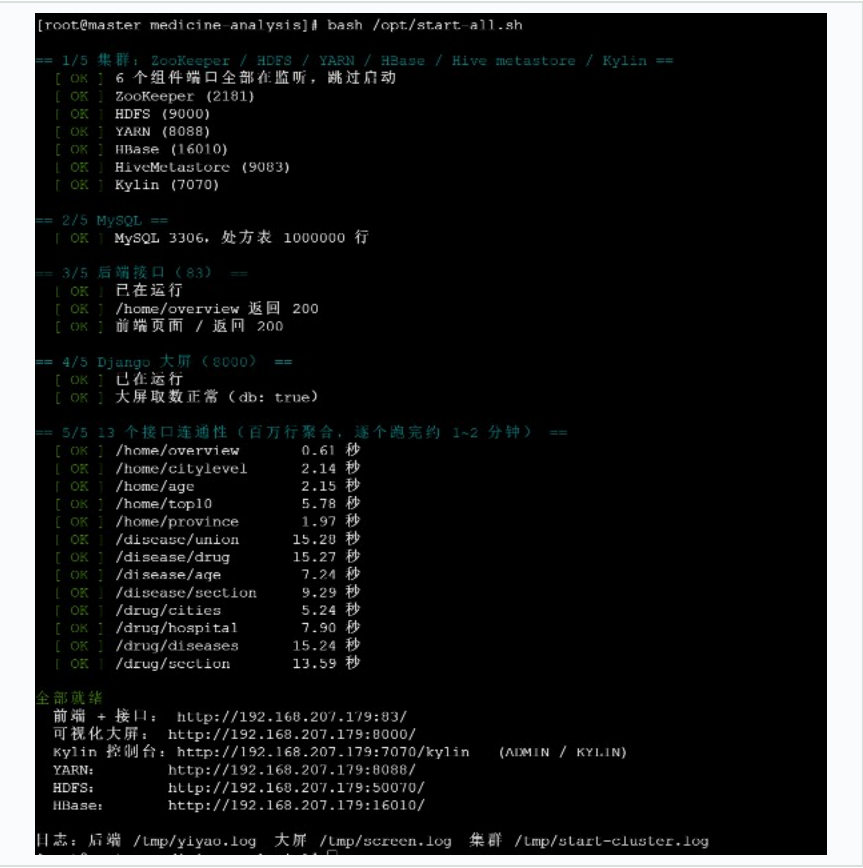


图 9 start-all.sh 五阶段输出与 13 个接口耗时

服务器终端实录， 2026-11-14

这份输出能逐项核对

阶段一	六个组件端口全部在监听，逐个打印 2181 到 7070
阶段二	MySQL 3306 在监听，处方表读到 1000000 行
阶段三	后端 /home/overview 与前端页面都返回 200
阶段四	Django 大屏取数正常，健康检查 db 为 true
阶段五	13 个接口逐个计时，最慢 /disease/union 15.26 秒

脚本末尾同时打印五个访问入口

主站 83 、大屏 8000 、 Kylin 7070 、 YARN 8088 、 HDFS 50070 、 HBase 16010

数据流向：一条请求从表到页面经过六步

每一步都标了真实的数据量与耗时，接口如何选路写在下半页

MySQL 业务表
六张表
100 万行

→

HDFS 与 Hive
bi_data/mysql
6 个目录

→

Kylin 预计算
256 个 cuboid
2.95 GB

→

agg 汇总表
七张表
9 个索引

→

13 个接口
0.61 至 15.26 秒
全部 200

→

三套页面
12 张图表
83 与 8000

接口取数的两条路径

路径一　命中 Cube　　**5 个接口**

取值少的维度直接组合，不查明细表

实测 0.61 到 2.19 秒

路径二　转 MySQL　　**8 个接口**

先查汇总表，再走索引关联维表取名称

实测 5.20 到 15.26 秒

页面怎么拿到这些数

- 接口返回统一包一层 code 与 data，前端按 data 取数，失败时给出错误提示
- 对象字段名统一转成首字母大写再下发，前端组件按固定字段名渲染，不用逐页适配
- 图表组件只做渲染，取数逻辑集中在每个组件的 load 方法，搜索时重新调用一次
- 前端超时 60 秒，超过才判定失败；这一条写在 api/index.js 的 FALLBACK_TIMEOUT

Kylin 建模与 Cube 构建

八列维度、两个度量， Cube 建成后页面聚合直接读它

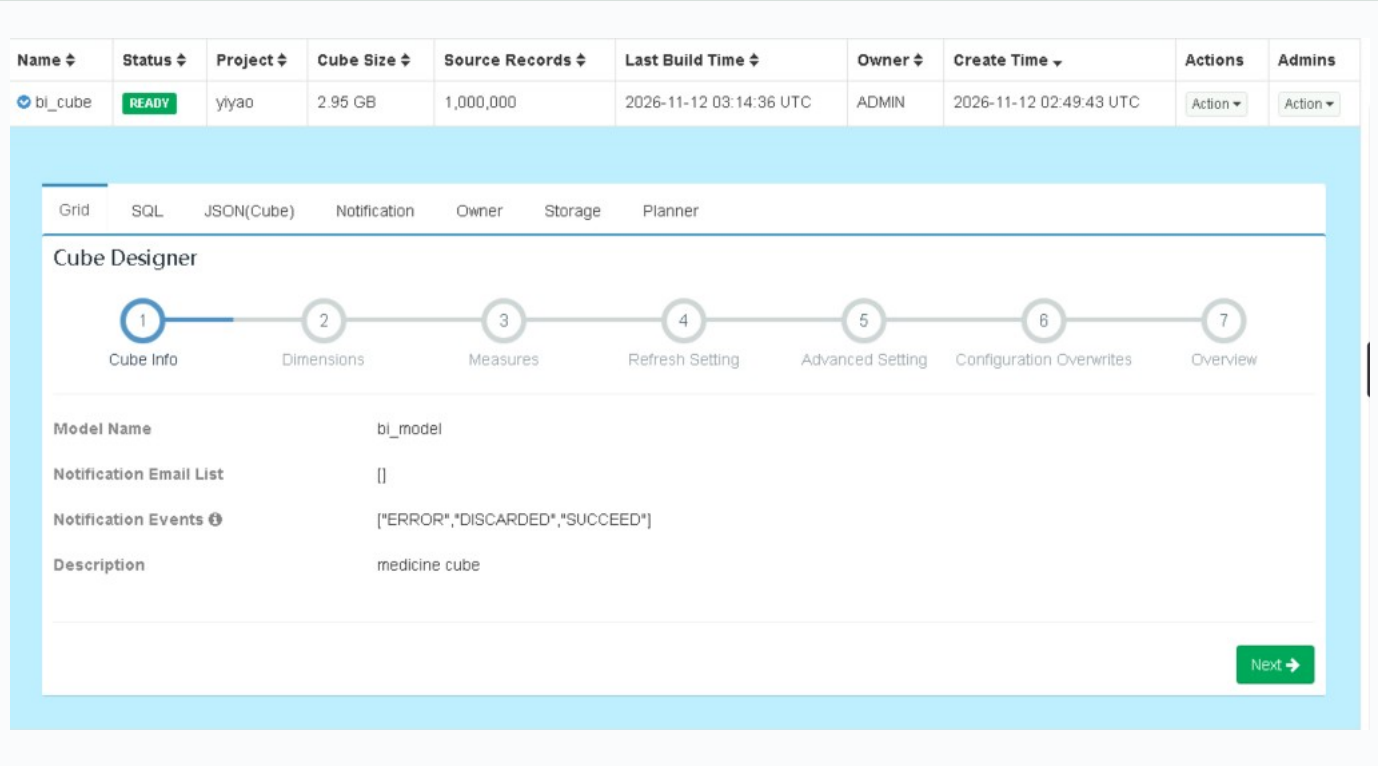


图 10　Kylin 的 bi_cube 列表： READY 、 2.95 GB 、源记录 100 万

Kylin 控制台截图， 2026-11-12

项目	取值
项目名	yiyao
Cube	bi_cube
状态	READY
创建时间	11-12 02:49 UTC
完成时间	11-12 03:14 UTC
体积	2.95 GB
源记录	1,000,000
Cuboid	256

为什么只放八列

维度每加一列， cuboid 翻倍。八列已是 256 个组合，构建二十多分钟；医院与科室基数上千，进 Cube 后构建时间不可接受，因此改走 MySQL 索引。

查询引擎切换与接口返回

业务 SQL 只维护一份，改写规则集中在网关；接口统一返回 code 与 data

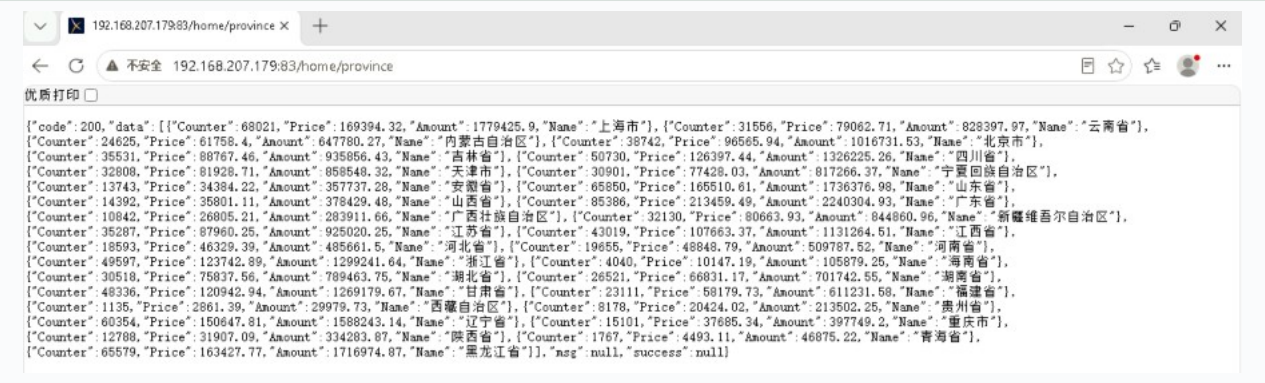


图 11 /home/province 的原始返回： code 200 与各省数值

改写规则五条

原写法	改写后
BIGDATA.PRESCRIPTION_DISEASES	prescription_diseases
M_DIAGNOSTIC	m_diagnostic1
PRESCRIPTION_DISEASES.AGE	m_age
M_DRUG = DRUG.DRUG	DRUG_CODE = DRUG.DRUG
VARCHAR	CHAR

配置项

```
query.engine = kylin | mysql
```

切换时业务代码不改动， Kylin 不可用时改一行配置回到 MySQL ；实测 13 个接口中 5 个由 Cube 直接回答， 8 个转 MySQL 索引关联。

返回结构带来的好处

code 为 200 表示成功， data 里是业务数组，前端不需要解析额外包装；字段名统一转成 Counter 、 Price 、 Amount 、 Name 这类首字母大写形式，十二张图共用同一套取数写法，新增图表只要换 url 与字段映射。

人员分工与进度安排

五天排期，每天有明确产出

成员	角色	承担工作	交付物
占竞	组长	架构设计、 Kylin 建模、后端接口	13 个接口与查询网关
李艺淼	数据	数据清洗、 Cube 构建、可视化大屏	bi_cube 与 Django 大屏
付悌安	前端	三套页面、图表组件、结构对比模块	12 张图表与对比表
杨博文	文档	需求规格、部署验收、数据集与基准值	五份文档
洪嘉伟	测试	接口性能、数据一致性、验收用例	测试用例与校验脚本

五天进度与当日产出

11-11
环境搭建
六组件端口全通

11-12
数据入仓
六张表一百万行

11-13
维度建模
bi_cube READY

11-14
接口与前端
13 接口与三页

11-15
联调验收
基准值与文档

完成情况与后续计划

已完成项均可复核，后续计划按优先级排列

已完成并可复核

- 六节点集群与 13 个接口在 192.168.207.179 上持续运行
- bi_cube 8 维 2 度量 256 个 cuboid，体积 2.95 GB
- 七张汇总表上线，最大 100 万行，索引 9 个
- 三套页面共 12 张图表，数据全部来自后端接口
- 五份交付文档与四个运维脚本，检查项可重复执行

后续计划

第一项

按科室维度扩展预聚合表，覆盖更多组合

第二项

接入按月增量构建，把单月数据扩展为多个月

第三项

13 个接口纳入自动化测试，改一次跑一遍

第四项

基准值校验接入启动脚本，每次部署自动比对

系统地址 http://192.168.207.179:83/　大屏 http://192.168.207.179:8000/　代码与文档已提交